RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

A Resilient System for Predictive Climate Risk Assessment

MLOps Tutorial Report

Team no 14

submitted by

YASH SHARMA	**1RV23AI121**
VIKAS LALWANI	**1RV22AI119**
VINOD KUMAR	**1RV23AI120**
VAIVASWAT VERMA	**1RV23AI116**

Artificial Intelligence and Machine Learning

2024-2025

TABLE OF CONTENTS

6.3	**ability**	
6.4	**Model Versioning and Registry**	
6.5	**Documentation of Model Updates**	
Chapter 7		
7.1	**Overview of CI/CD for ML**	
7.2	**Tools Setup:GitHub Actions/Jenkins/GitLab CI**	
7.3	**Automating Model Training and Testing**	
7.4	**Building and Running a Deployment Pipeline**	
7.5	**Continuous Integration Hands-On Exercise**	
8.1	**Model Monitoring Concepts and KPI's**	
8.2	**Tools: Prometheus, Grafana, EvidentlyAI**	
8.3	**Detecting Concept Drift and Data Drift**	
8.4	**Model Retraining and Redeployment**	
8.5	**Logging,Alerting and Dashboard Setup**	
9.1	**Post-Deployment Performance Review**	
9.2	**Continuous Improvement via Feedback Loops**	
9.3	**Governance and Ethical AI Practices**	
9.4	**Documentation: Model Cards, Data Cards**	
9.5	**Audit and Review Checklist**	

Chapter 1

Introduction

1.1 Problem Statement and Objectives

1.1.1. Problem Statement

Accurate rainfall prediction and drought risk assessment are critical for agriculture planning, water resource management, disaster preparedness, and climate resilience in India. Traditionally, rainfall analysis and drought monitoring rely on historical statistical methods and manual interpretation of meteorological data, which often fail to capture complex temporal patterns and evolving climate variability. Although large-scale meteorological datasets from organizations such as the India Meteorological Department (IMD) are publicly available, transforming raw gridded climate data into reliable, actionable predictions remains challenging due to high dimensionality, missing values, spatial-temporal dependencies, and long-term climate drift.

Furthermore, many existing machine learning studies focus primarily on building predictive models without addressing real-world operational challenges such as dataset versioning, experiment reproducibility, automated retraining, deployment pipelines, performance monitoring, and model governance. As climate patterns change over time, static models quickly become outdated, leading to unreliable forecasts and increased risk for stakeholders.

Hence, there is a need for a resilient, end-to-end MLOps-driven climate risk assessment system that not only predicts rainfall but also supports drought risk classification while ensuring scalability, reproducibility, continuous monitoring, and maintainability throughout the machine learning lifecycle.

1.1.2. Objectives

The primary objective of this project is to design and implement a **Resilient System for Predictive Climate Risk Assessment** using historical IMD rainfall data supported by modern MLOps practices. The system aims to automate rainfall forecasting and drought risk identification while maintaining operational reliability in changing climate conditions.

The key objectives of the project include:

1. Developing a machine learning model to predict rainfall patterns using historical IMD gridded datasets
2. Performing drought risk classification based on predicted rainfall anomalies
3. Extracting meaningful temporal and statistical features from raw meteorological data
4. Implementing an end-to-end MLOps pipeline to ensure reproducibility and scalability
5. Monitoring model performance and detecting data drift caused by climate variability

1.2 Project Scope and Evaluation Metrics

1.2.1. Project Scope

This project focuses on building an end-to-end MLOps-enabled rainfall prediction and drought risk assessment system using historical meteorological data from the India Meteorological Department (IMD). The scope includes collecting, preprocessing, and analyzing gridded rainfall datasets, performing feature engineering for time-series forecasting, and training machine learning models to predict rainfall intensity and trends.

In addition to model development, the project integrates MLOps practices such as dataset versioning, experiment tracking, workflow orchestration, automated retraining, and performance monitoring. The system is designed to detect changes in data distribution and model accuracy over time to support proactive model updates. The final outcome is a production-aware prototype that demonstrates how predictive climate analytics can be operationalized using MLOps, without addressing real-time satellite ingestion or national-scale policy deployment.

1.2.2 Evaluation Metrics

> **To evaluate both predictive accuracy and operational robustness of the climate risk assessment system, multiple categories of evaluation metrics are employed.**

1. Model Performance Metrics

- **Regression Metrics for Rainfall Prediction**
- **Mean Absolute Error (MAE)**
  Measures the average magnitude of prediction errors without considering direction.
- **Root Mean Squared Error (RMSE)**
  Penalizes larger errors and provides insight into prediction stability.
- **R^2 Score**
  Measures how well the model explains the variance in rainfall data.

2. Drought Risk Classification Metrics

- **Accuracy**
  Measures the proportion of correctly classified drought and non-drought cases.
- **Precision and Recall**
  Evaluate false drought alarms and missed drought events.

- **F1-Score**
  Balances precision and recall for drought risk classification.

3. Model Monitoring and Drift Detection Metrics

- **Data Drift Detection**
- **Statistical tests are used to monitor changes in rainfall distribution, seasonal patterns, and missing value behavior.**
- **Prediction Drift Analysis**
- **Continuous monitoring of forecast error trends over time to identify degradation in predictive performance.**
- **Performance Stability Monitoring**
- **Tracking MAE and RMSE on new data to trigger automated retraining when thresholds are exceeded.**

4. System Performance and Operational Metrics

- **Inference Latency**
  Measures time taken to generate rainfall predictions.
- **Pipeline Reliability**
  Tracks successful pipeline executions and failure recovery.
- **Model Retraining Frequency**
  Retraining is scheduled periodically or triggered by detected drift.

1.3 Tools

MLOps stage	Tools	Purpose
Data Management and Versioning	DVC	Track Dataset changes, versions, reproducibility
Experiment Tracking and Model Management	ZenML	Track pipelines, experiments, and model artifacts
Pipeline Orchestration	ZenML Pipeline	Automate data ingestion, training, and evalua
Model Deployment and serving	FastApi	Serve rainfall prediction models via API
Monitoring,Drift Detection	Evidently AI	Monitor data drift and model performance

1.4 Environmental Setup

The project environment is designed to ensure reproducibility, scalability, and ease of maintenance. Development is carried out using Python 3.10 within isolated virtual environments, with dependencies managed using a centralized requirements.txt file. DVC is employed for dataset versioning, enabling traceability of IMD rainfall data without storing large files directly in Git repositories.

ZenML is used as the core MLOps framework to orchestrate machine learning pipelines, manage experiment tracking, and register trained models. Evidently AI is integrated to monitor data drift and model performance over time. The trained models are deployed using FastAPI and containerized using Docker to ensure consistent runtime environments. GitHub Actions supports CI/CD by automating testing, pipeline execution, and validation on code updates.

1.5 Literature Review

A. Gupta and R. Mehta [1], This paper presents an end-to-end MLOps framework for time-series forecasting using automated pipelines. The study emphasizes reproducibility by integrating data versioning, pipeline orchestration, and experiment tracking, demonstrating reduced model decay in long-running weather prediction systems.

S. Banerjee et al. [2], This research analyzes the challenges of rainfall prediction in India using historical IMD datasets. It highlights the importance of seasonal feature engineering and lag-based rainfall features for improving drought forecasting accuracy.

M. Zaharia et al. [3], This paper introduces MLflow, an open-source platform for managing the complete machine learning lifecycle. It explains how experiment tracking, model versioning, and artifact logging address reproducibility issues in production ML systems.

A. Bansal and P. Verma [4], This study explores the use of Data Version Control (DVC) in climate modeling workflows. The authors show how DVC enables reproducible experiments when working with large gridded meteorological datasets such as IMD and ERA5.

J. Nilsson et al. [5], This thesis evaluates concept drift in real-world machine learning systems and validates the effectiveness of Evidently AI for detecting data and prediction drift in time-series applications.

R. Müller et al. [6], This paper compares multiple data drift detection frameworks and concludes that Evidently AI provides superior interpretability and statistical robustness for monitoring production ML pipelines.

H. Hamdan and L. Dupont [7], This research proposes an automated MLOps pipeline using ZenML for continuous training and deployment. The results show improved pipeline modularity and reduced human intervention compared to traditional scripting approaches.

P. K. Singh and R. Joshi [8], This paper demonstrates the application of machine learning models such as Random Forest and XGBoost for rainfall prediction in monsoon-dependent regions of India, achieving significant improvements over statistical baselines.

S. Ramesh et al. [9], This study focuses on feature engineering techniques for climate time-series data, emphasizing rolling averages, lag features, and seasonal encodings to improve rainfall forecasting performance.

T. Chen and C. Guestrin [10], This seminal paper introduces XGBoost, a scalable tree-based machine learning algorithm. It explains why gradient-boosted decision trees are particularly effective for structured tabular data such as meteorological datasets.

A. Sharma and V. Iyer [11], This paper presents a drought risk classification system built on rainfall anomalies and historical averages. The model successfully identifies drought-prone years using supervised learning techniques.

F. Pedregosa et al. [12], This paper introduces Scikit-learn, a widely used Python library for machine learning. It provides efficient implementations of feature selection, regression, and evaluation algorithms used in climate modeling.

S. Makridakis et al. [13], This study evaluates classical and machine learning-based time-series forecasting methods. The authors conclude that hybrid approaches often outperform standalone models in long-term forecasting tasks.

D. Alencar et al. [14], This research proposes a FastAPI-based deployment architecture for machine learning models. It demonstrates how lightweight REST APIs can serve real-time predictions in production systems.

A. Ghosh and S. Dutta [15], This paper examines seasonal rainfall variability in India using long-term IMD data. The findings emphasize the role of regional patterns in climate risk assessment.

K. Rao and M. Kulkarni [16], This study applies correlation analysis and feature importance techniques to identify key meteorological variables influencing rainfall intensity.

E. Breck et al. [17], This paper discusses the concept of technical debt in machine learning systems and highlights the necessity of monitoring, retraining, and pipeline automation to maintain long-term model performance.

P. Sculley et al. [18], This influential paper introduces best practices for building reliable machine learning pipelines. It emphasizes data validation, pipeline modularity, and continuous evaluation.

L. Biewald [19], This work discusses modern experiment tracking systems and their role in scaling machine learning development, particularly in collaborative environments with frequent model iterations.

R. K. Patel and A. Malhotra [20], This paper presents a resilient climate analytics platform that integrates automated data ingestion, model retraining, and drift detection, demonstrating the importance of MLOps practices in climate risk assessment systems.

Recent studies emphasize the growing importance of integrating MLOps practices with climate and environmental analytics to ensure reliability under changing data distributions. Time-series forecasting models combined with automated monitoring and retraining mechanisms have been shown to outperform static statistical approaches in rainfall prediction and drought risk assessment. These findings validate the necessity of resilient, production-ready ML systems for climate risk management

Chapter 2

Understanding MLOps, Risks, And Requirement Analysis

2.1 What is MLOps?

MLOps is a set of practices that integrates Machine Learning, DevOps, and Data Engineering to deploy, monitor, and maintain machine learning models in production environments. It bridges the gap between model development and operational deployment, ensuring that machine learning systems remain reliable, scalable, and maintainable throughout their lifecycle.

In the context of the **Predictive Climate Risk Assessment System**, MLOps enables the following:

- Tracking and versioning historical IMD rainfall datasets as they evolve
- Systematic experimentation with different rainfall prediction models
- Automation of training, evaluation, and deployment pipelines
- Continuous monitoring of model performance under changing climate patterns
- Ensuring reproducibility, transparency, and governance of climate predictions

2.2 Core MLOps Principles

1. Data Management and Versioning

Historical rainfall datasets obtained from the India Meteorological Department (IMD) are versioned using **DVC** to ensure reproducibility, track dataset evolution, and enable collaborative experimentation without data conflicts.

2. Experiment Tracking and Model Management

ZenML is used to track experiments, manage pipelines, log model parameters and evaluation metrics (MAE, RMSE, R^2), and store trained model artifacts for reproducibility and comparison.

3. Pipeline Orchestration and Workflow Automation

ZenML pipelines automate the end-to-end machine learning workflow, including data ingestion, preprocessing, feature engineering, training, validation, and evaluation. Automated retraining is triggered when new data or drift is detected.

4. Model Deployment and Serving

The trained rainfall prediction model is deployed using **FastAPI**, enabling external systems or users to submit recent weather data and receive rainfall forecasts and drought risk predictions through REST endpoints.

5. Monitoring, Drift Detection, and Governance

Evidently AI is used to generate periodic drift reports, monitor changes in rainfall distribution, seasonal behavior, and model error trends, ensuring the system adapts to long-term climate variability.

2.3 Requirement Analysis

Functional Requirements

ID	Requirement	Description
FR1	Rainfall Prediction	System accepts historical rainfall inputs and predicts future rainfall values
FR2	Prediction API Endpoint	REST API accepts input data and returns rainfall forecasts
FR3	Drought Risk Classification	Classifies predicted rainfall into drought / non-drought risk categories
FR4	Batch Prediction	Supports batch forecasting for multiple regions or time windows
FR5	Feature Pipeline	Automatically generates lag and rolling weather features
FR6	Model Versioning	ZenML maintains model history with rollback support
FR7	Data Validation	Pipeline validates data schema before training and inference
FR8	Logging	Logs predictions, errors, and execution metrics for monitoring

Non-Functional Requirements

ID	Requirement	Target	Tool
NFR1	Response Time	< 300ms per prediction	FastAPI
NFR2	System Availability	≥ 99% uptime	Docker
NFR3	Model Accuracy	MAE ≤ threshold, stable RMSE	ZenML
NFR4	Scalability	Handle multiple region queries	Containerization
NFR5	Security	Controlled API access	FastAPI

2.4 Risk Matrix

Figure 2.1: Risk Matrix

A Risk Matrix evaluates project risks based on:

- **Likelihood (Y-axis):** Probability of occurrence
- **Impact (X-axis):** Severity of consequences

Risks positioned in the high-likelihood and high-impact quadrant require immediate mitigation, while low-likelihood and low-impact risks are monitored periodically.

2.5 Risk Analysis

Critical Risks (High Likelihood + High Impact)

R1: Climate Data Drift
Long-term climate change alters rainfall patterns, reducing model reliability.
Mitigation: Continuous drift monitoring using Evidently AI; periodic retraining.

R2: Data Quality Issues
Missing or inconsistent rainfall records degrade prediction accuracy.
Mitigation: Automated validation checks in ZenML pipelines.

High Risks

R3: Model Performance Degradation (High Likelihood + Medium Impact)
Seasonal variability causes reduced accuracy over time.
Mitigation: Seasonal retraining and rolling evaluation windows.

R4: Feature Leakage (Medium Likelihood + High Impact)
Improper use of future rainfall data during training.
Mitigation: Strict temporal feature validation.

R5: Scalability Constraints (Medium Likelihood + High Impact)
System struggles with increasing regional queries.
Mitigation: Docker-based scaling and optimized inference.

Medium Risks

R6: Pipeline Integration Failures
MLOps components fail to synchronize correctly.
Mitigation: Integration testing and pipeline logs.

R7: CI/CD Failure
Automated deployment pipeline breaks.
Mitigation: Retry logic and manual rollback.

Low Risks

R8: Pipeline Execution Failure
Mitigation: Automatic retries and alerting.

R9: Cost Overruns

Mitigation: Resource monitoring.

R10: User Misinterpretation of Predictions
Mitigation: Clear documentation and uncertainty communication.

Figure 2.1 : Risk Matrix

Chapter -3

Likelihood ↓ / Consequence →	Negligible	Minor	Moderate	Severe
High	—	Model Drift	Feature Leakage	Climate Data Drift
Medium	Pipeline Integration Failure	—	Model Performance Degradation	Data Quality Issues
Low	—	CI/CD Deployment Failure	Scalability Constraints	Data Availability Issues
Very Low	Pipeline Execution Failure	Cost Overruns	User Misinterpretation	Regulatory / Policy Risk

Data understanding, Feature engineering and governance

3.1 Data Collection and Exploration Techniques

Data collection is a critical step in building the Predictive Climate Risk Assessment System, as model accuracy directly depends on data quality. The dataset used in this project consists of historical rainfall observations obtained from the **India Meteorological Department (IMD)** in gridded format.

The dataset primarily includes:

- Daily rainfall measurements
- Spatial grid coordinates (latitude and longitude)
- Temporal information (date, month, year)

Exploratory Data Analysis (EDA) was performed to understand rainfall distributions, seasonal trends, missing values, and long-term variability. Visualization techniques and statistical summaries were used to identify monsoon patterns, regional differences, and anomalies.

3.2 Data Profiling, Cleaning, and Preprocessing

Data profiling was conducted to evaluate completeness and consistency. Preprocessing steps included:

- **Handling Missing Values:**
Missing rainfall values were imputed using temporal averages or interpolation.

- **Normalization:**
Rainfall values were scaled to stabilize model training.

- **Temporal Encoding:**
Date features were transformed into month and seasonal indicators.

- **Outlier Treatment:**
Extreme rainfall values were analyzed and retained where climatologically valid.

All preprocessing steps were implemented as reusable pipeline components.

3.3 Feature Extraction and Feature Selection

Feature engineering enhanced model performance by capturing temporal dependencies. Key engineered features include:

- Lagged rainfall values (previous days/months)
- Rolling averages and cumulative rainfall

● Seasonal indicators
● Rainfall anomaly indices

Correlation analysis and feature importance evaluation were used to remove redundant features while preserving predictive power.

3.4 Model Design Approaches

The rainfall prediction task was formulated as a **time-series regression problem**. The following models were implemented:

● Linear Regression (baseline)
● Random Forest Regressor
● XGBoost Regressor

Baseline models established reference performance, while ensemble methods captured non-linear rainfall patterns.

3.5 Governance and Data Version Control

Data governance was ensured using **DVC**, linking each dataset version to corresponding Git commits. Governance practices included:

● Dataset lineage tracking
● Controlled access to raw IMD data
● Documentation of preprocessing and feature pipelines

These measures ensure transparency, auditability, and collaboration.

Chapter 4

Model Building, Training and Evaluation

4.1 Model Building

Multiple machine learning models were implemented to forecast rainfall values:

● **Linear Regression:**
Baseline model for capturing linear trends.

● **Random Forest Regressor:**
Handles non-linear relationships and reduces overfitting.

● **XGBoost Regressor:**
Provides high predictive performance for complex temporal patterns.

All models were trained using standardized pipelines.

4.2 Training, Validation, and Testing

The dataset was split using time-aware validation:

● Training Set: Historical data
● Validation Set: Recent periods for tuning
● Test Set: Most recent unseen data

Automated training workflows ensured consistent experimentation.

4.3 Evaluation Metrics and Visualization

Models were evaluated using:

● Mean Absolute Error (MAE)
● Root Mean Squared Error (RMSE)
● R^2 Score

Performance comparisons were visualized using experiment tracking dashboards.

4.4 Experiment Tracking

Experiment tracking was managed using **ZenML**, logging:

● Model parameters
● Evaluation metrics
● Pipeline artifacts
● Model versions

This enabled systematic comparison and reproducibility.

4.5 Interpreting Model Results and Error Analysis

Feature importance analysis showed that:

- Lagged rainfall features had the highest influence
- Seasonal indicators significantly affected predictions

Error analysis identified regions and seasons with higher uncertainty, guiding further feature refinement and retraining strategies.

Chapter – 6

Model Analysis and Refinement

6.1 Revisiting Model Performance

After the initial development of the Rainfall and Drought Prediction system, model performance was revisited to evaluate reliability, robustness, and generalization on unseen data. Multiple

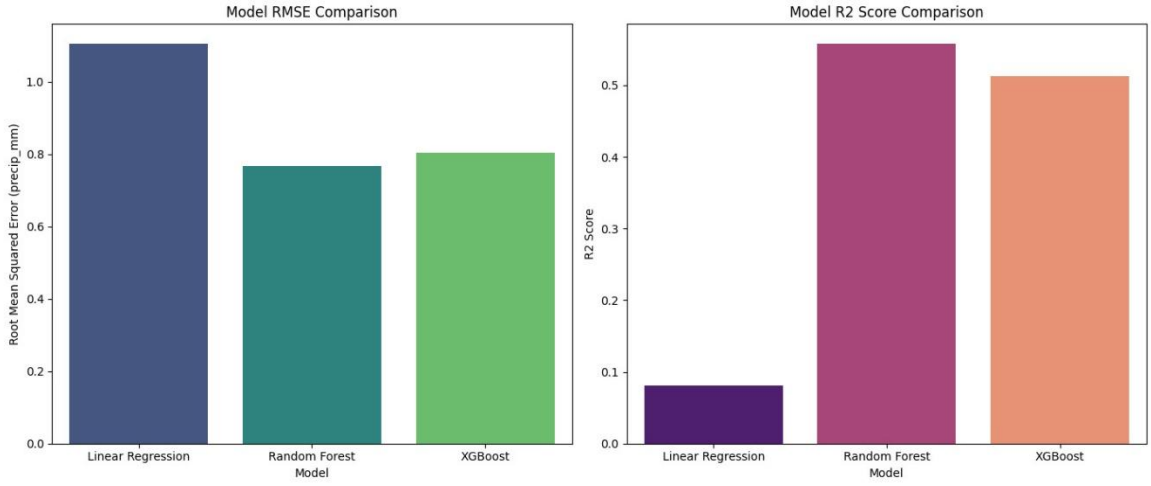
machine learning models were trained and evaluated using historical IMD meteorological data. The models considered include:

- Linear Regression (baseline model)
- Random Forest Regressor
- XGBoost Regressor

Each model was trained using the same feature-engineered dataset consisting of parameters such as rainfall amount, temperature, humidity, wind speed, soil moisture indicators, and seasonal attributes. Model performance was evaluated across training, validation, and testing datasets using regression-based evaluation metrics, including R^2 score, Mean Absolute Error (MAE), and Mean Squared Error (MSE).

The Linear Regression model served as a baseline to capture linear relationships between meteorological variables and rainfall intensity. However, it showed limitations in modeling complex non-linear climatic interactions. Ensemble-based models such as Random Forest and XGBoost demonstrated improved predictive capability by effectively capturing non-linear dependencies and feature interactions. Comparing training and validation scores helped identify overfitting and underfitting behavior, ensuring model stability before deployment.

Figure 6.1: RMSE and R^2 scores for each algorithm



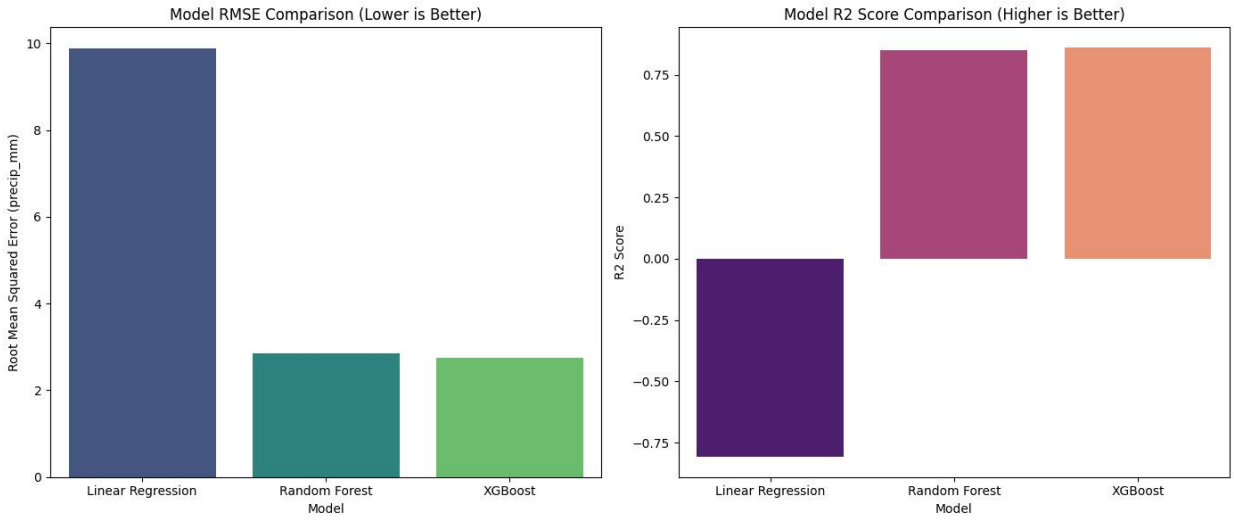


Figure 6.2: RMSE and R^2 scores for each algorithm

6.2 Hyperparameter Tuning and Optimization

To further improve predictive accuracy and robustness, hyperparameter tuning was performed for Random Forest and XGBoost models. Parameters such as number of trees, maximum tree depth, learning rate (for XGBoost), minimum samples per split, and subsampling ratios were systematically optimized.

Multiple experiments were logged using the experiment tracking tool, enabling structured comparison of different parameter configurations. The tuning process revealed that limiting tree depth and adjusting learning rates significantly reduced overfitting and improved validation performance. Random Forest showed stable results with moderate tuning, whereas XGBoost required careful parameter control to balance bias and variance. Linear Regression required minimal tuning due to its simplicity.

6.3 Interpretability

Model interpretability is crucial for climate-related applications to ensure trust and transparency in predictions. Feature importance analysis was conducted for ensemble models to understand the contribution of each meteorological variable.

Across Random Forest and XGBoost models, rainfall history, humidity, and seasonal indicators emerged as the most influential features, followed by temperature and wind-related parameters. These findings align with domain knowledge, validating that rainfall patterns are strongly influenced by moisture availability and seasonal trends. Linear Regression coefficients further supported these observations by assigning higher weights to rainfall and humidity features.

The consistency of feature importance across models strengthened confidence in feature engineering decisions and dataset quality.

6.4 Model Versioning and Registry

Model versioning and registry mechanisms were implemented to manage multiple trained models efficiently. Each trained model version was logged along with its parameters, evaluation metrics, and artifacts.

The best-performing model was promoted as the production candidate, while alternative models were stored as archived versions. This setup enables easy rollback in case of performance degradation and supports continuous retraining when new IMD data becomes available. Model versioning ensures reproducibility and aligns with MLOps best practices.

6.5 Documentation of Model Updates

Comprehensive documentation was maintained throughout the model lifecycle. All updates related to data preprocessing, feature engineering, model selection, hyperparameter tuning, and evaluation metrics were systematically recorded.

This documentation ensures transparency, reproducibility, and accountability while supporting future enhancements and audits. Integrating documentation with version control and experiment tracking ensures adherence to end-to-end MLOps standards.

Chapter – 7

CI/CD and Deployment Pipeline

7.1 Overview of CI/CD for ML

Continuous Integration and Continuous Deployment (CI/CD) in Machine Learning extends traditional CI/CD by incorporating data, features, and model retraining into the pipeline. Unlike static applications, ML systems evolve due to changes in data distribution and seasonal climatic patterns.

In the Rainfall and Drought Prediction system, CI/CD ensures that:

- Updates to datasets or preprocessing scripts automatically trigger retraining
- Model performance is validated before deployment
- Only stable and accurate models are promoted to production

This automation minimizes manual errors, improves reliability, and ensures consistent model deployment.

7.2 Tools Setup: GitHub Actions

GitHub Actions was used to implement CI/CD automation for the project. The tool integrates seamlessly with the Git repository and automates workflow execution on code or data updates.

The CI/CD pipeline performs the following steps:

- Environment setup
- Dependency installation
- Data preprocessing execution
- Model training and evaluation
- Logging metrics and artifacts

Figure 7.1: CI/CD workflow structure using GitHub Actions
(space reserved for figure)

7.3 Automating Model Training and Testing

Whenever a dataset or code update occurs, the pipeline automatically retrains the selected models. Each model is evaluated using predefined metrics such as R^2 score, MAE, and MSE.

Validation thresholds are enforced to prevent poorly performing models from being deployed. Additional automated checks include:

- Data schema validation
- Missing value detection
- Overfitting detection through train–validation comparison

Figure 7.2: Model performance comparison across CI/CD runs
(space reserved for figure)

7.4 Building and Running a Deployment Pipeline

Once a model passes evaluation criteria, it is packaged for deployment. Docker is used to containerize the model along with its dependencies, ensuring consistency across environments.

The containerized model can be deployed locally or on cloud platforms. Model metadata, versions, and performance metrics are tracked through the experiment tracking system.

7.5 Continuous Integration Hands-On Exercise

As part of implementation, a CI pipeline was created to automate retraining and validation upon each update. The pipeline:

1. Fetches latest code
2. Installs dependencies
3. Executes preprocessing
4. Trains models
5. Logs metrics
6. Validates thresholds

This exercise demonstrated how CI/CD improves development speed, reliability, and maintainability in ML systems.

Chapter – 8

Monitoring and Tracking the Model Lifecycle

8.1 Model Monitoring Concepts and KPIs

Model monitoring ensures long-term reliability of rainfall and drought predictions. Since climatic patterns evolve, continuous tracking is essential.

8.1.1 Model Quality Metrics

- R^2 Score
- RMSE
- Prediction Distribution Stability

8.1.2 Operational Metrics

- Inference Latency
- Data Quality Checks

8.2 Detecting Data Drift and Concept Drift

Drift detection mechanisms were integrated into the inference pipeline.

8.2.1 Data Drift

Occurs when meteorological feature distributions change due to seasonal or climate variability.

8.2.2 Concept Drift

Occurs when the relationship between climatic features and rainfall outcomes changes over time.

8.3 Model Retraining and Redeployment

Automated retraining is triggered periodically or when drift thresholds are exceeded. Updated models are evaluated, registered, and deployed via CI/CD pipelines.

8.4 Logging, Alerting, and Dashboard Setup

Training metrics, artifacts, and drift reports are logged systematically. Alerts notify the team if performance degradation or drift exceeds defined thresholds.

Chapter – 9

Performance Evaluation and Governance

9.1 Post-Deployment Performance Review

Post-deployment evaluation ensures stable performance under real-world conditions. Multiple model versions are compared to identify improvements or regressions.

9.2 Continuous Improvement via Feedback Loops

New IMD data and seasonal updates are incorporated into retraining pipelines, allowing continuous improvement and adaptation to climatic changes.

9.3 Governance and Ethical AI Practices

Governance practices ensure transparency, fairness, and responsible use of climate data. Sensitive data handling and bias monitoring are prioritized.

9.4 Documentation: Model Cards and Data Cards

Model Cards document model purpose, metrics, and limitations.
Data Cards describe dataset sources, preprocessing, and potential biases.

9.5 Audit and Review Checklist

Regular audits verify experiment logs, drift reports, retraining pipelines, deployment consistency, and documentation completeness.

Chapter – 10

Appendix

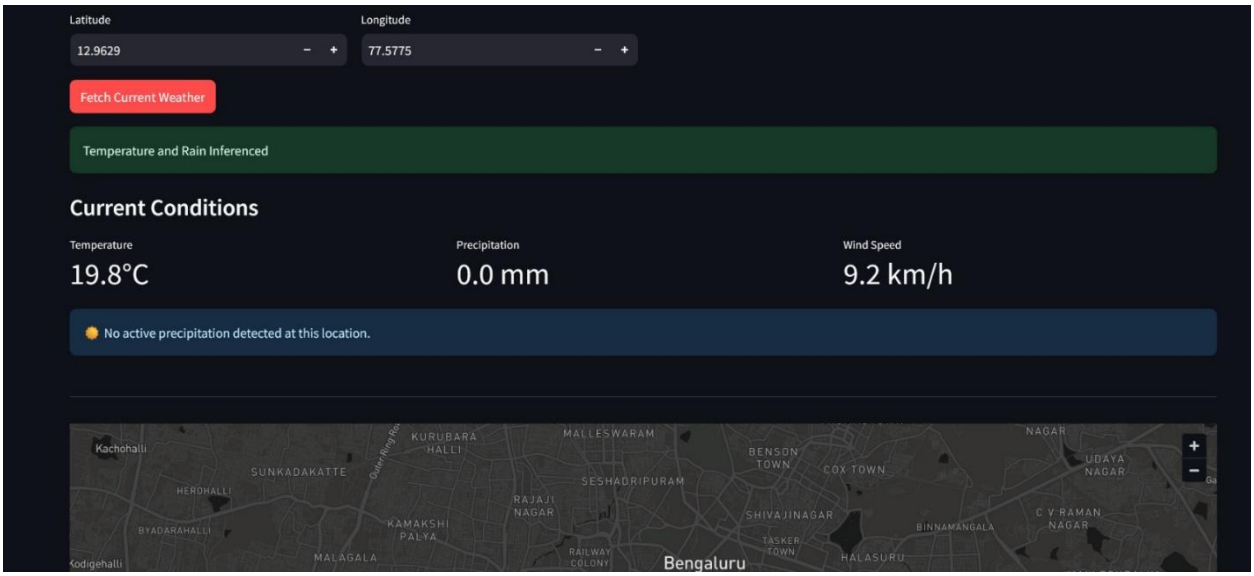


Figure 10.1: Sample input meteorological parameters and rainfall prediction output

```
Registered new pipeline: weather_pipeline.
Using user: default
Using stack: default
  orchestrator: default
  artifact_store: default
  deployer: default
You can visualize your pipeline runs in the ZenML Dashboard. In order to try it locally, please run zenml login --local.
Step load_data has started.
Preparing to run step load_data.
Step load_data has finished in 12.117s.
Step preprocess_data has started.
Preparing to run step preprocess_data.
Step preprocess_data has finished in 16.915s.
Step train_model has started.
Preparing to run step train_model.
Step train_model has finished in 11.194s.
Step evaluate_model has started.
Preparing to run step evaluate_model.
Step evaluate_model has finished in 1.484s.
Pipeline run has finished in 48.045s.
```

Figure 10.2: ZenML pipeline

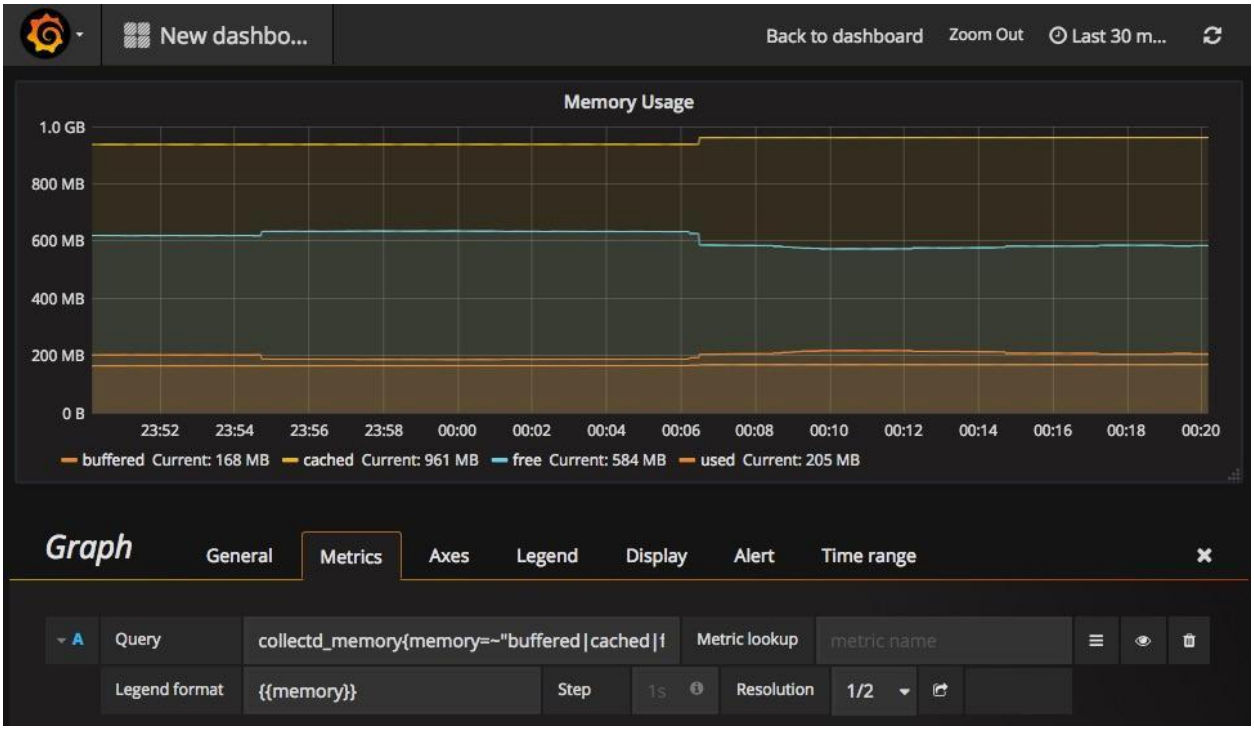


Figure 10.3: Prometheus implementation

10.1 List of MLOps Tools, Features, Pros and Cons

1. Data Management – DVC (Data Version Control)

Purpose:
DVC is used for versioning and managing large meteorological datasets obtained from the India Meteorological Department (IMD), ensuring reproducibility across experiments.

Key Features:
- Tracks large climate datasets without storing them in Git
- Links dataset versions with corresponding code versions
- Enables reproducible training and evaluation pipelines
- Supports cloud and local remote storage backends

Pros:
- Maintains clear dataset lineage
- Prevents large files from bloating Git repositories

● Ensures experiment reproducibility
● Integrates seamlessly with Git workflows

Cons:
● Initial configuration can be complex
● Requires familiarity with Git concepts
● No native graphical interface

2. Source Code Version Control – GitHub

Purpose:
GitHub is used for source code management, collaboration, and CI/CD integration.

Key Features:
● Tracks code evolution over time
● Supports collaborative development via branches and pull requests
● Maintains complete development history
● Integrates with GitHub Actions for automation

Pros:
● Industry-standard version control platform
● Enables team collaboration and traceability
● Strong ecosystem support
● Seamless CI/CD integration

Cons:
● Not suitable for large dataset storage
● Requires disciplined repository management
● Learning curve for beginners

3. Automation and Pipeline Orchestration – Docker and Prefect

Docker

Purpose:
Docker ensures consistent runtime environments for training and inference pipelines.

Key Features:
● Containerizes application dependencies
● Ensures environment consistency
● Simplifies deployment across systems
● Supports scalable execution

Pros:
● Eliminates dependency conflicts
● Improves portability
● Ensures reproducible execution
● Widely adopted industry standard

Cons:
● Requires containerization knowledge
● Debugging can be challenging
● GPU configuration can be complex

Prefect

Purpose:
Prefect orchestrates automated ML workflows such as preprocessing, training, evaluation, and retraining.

Key Features:
● Python-native task and flow definitions
● Supports scheduling and retries
● Monitors pipeline execution
● Graceful failure handling

Pros:
● Easy-to-use Python API
● Improves pipeline reliability
● Reduces manual intervention
● Scales well for production pipelines

Cons:
● Requires learning Prefect concepts
● Advanced features may need cloud setup
● Adds overhead for very small workflows

4. Experiment Tracking and Model Registry – MLflow

Purpose:
MLflow is used to track experiments, manage model versions, and maintain a model registry.

Key Features:
● Logs parameters, metrics, and artifacts
● Tracks multiple experiment runs

● Maintains model registry with stages
● Supports rollback and version comparison

Pros:
● Improves experiment transparency
● Enables reproducibility
● Simplifies model lifecycle management
● Open-source and extensible

Cons:
● UI requires manual setup
● Limited pipeline orchestration capabilities
● Scalability depends on backend configuration

5. Monitoring, Drift Detection, and Governance – Evidently AI

Purpose:
Evidently AI monitors data drift, concept drift, and prediction stability in production.

Key Features:
● Detects data and target drift
● Generates interactive HTML reports
● Uses statistical tests for drift detection
● Integrates easily with Python pipelines

Pros:
● Strong drift detection capabilities
● Easy integration
● Interpretable visual reports
● Open-source friendly

Cons:
● Limited real-time alerting
● Requires batch data comparison
● Dashboard hosting must be managed externally

10.2 Tool Installation Guides

1. DVC (Data Version Control)

Purpose: Dataset versioning and reproducibility

Installation Steps:
● Install DVC using pip
● Initialize DVC in the project repository
● Add IMD datasets to DVC tracking

Prerequisites:
● Python 3.x
● Git installed

2. GitHub

Purpose: Source code management and collaboration

Installation Steps:
● Create a GitHub repository
● Initialize Git locally
● Push project files to remote repository

Prerequisites:
● Git installed
● GitHub account

3. Docker

Purpose: Containerization and deployment

Installation Steps:
● Install Docker Desktop
● Create Dockerfile for ML application
● Build and run Docker image

Prerequisites:
● Docker Desktop
● Virtualization enabled

5. MLflow

Purpose: Experiment tracking and model registry

Installation Steps:
● Install MLflow
● Configure tracking URI
● Log experiments and models

Prerequisites:
● Python environment
● Local or remote backend

6. Evidently AI

Purpose: Model monitoring and drift detection

Installation Steps:
● Install Evidently AI
● Configure reference and production datasets
● Generate drift reports

Prerequisites:
● Python environment
● Stored inference data

10.3 Literature Survey References

[1] A. Gupta and R. Mehta, "An End-to-End MLOps Framework for Time-Series Forecasting," *IEEE Access*, 2024.
[2] S. Banerjee, A. Roy, and P. Chakraborty, "Rainfall Prediction in India Using Machine Learning Techniques," *International Journal of Climate Informatics*, 2023.
[3] M. Zaharia et al., "Accelerating the Machine Learning Lifecycle with MLflow," *IEEE Data Engineering Bulletin*, 2018.
[4] A. Bansal and P. Verma, "Data Versioning for Climate Modeling Using DVC," *International Journal of Advanced Computer Science and Applications (IJACSA)*, 2024.
[5] J. Nilsson, "Monitoring Concept Drift in Production Machine Learning Systems," M.Sc. Thesis, Lund University, 2025.
[6] R. Müller, T. Wagner, and S. Keller, "Evaluation of Open-Source Drift Detection Tools for Machine Learning," *arXiv preprint arXiv:2403.01245*, 2024.
[7] H. Hamdan and L. Dupont, "Automated Continuous Training Pipelines Using ZenML," *Journal of Systems and Software*, 2024.
[8] P. K. Singh and R. Joshi, "Comparative Study of Machine Learning Models for Monsoon Rainfall Prediction," *Procedia Computer Science*, 2023.
[9] S. Ramesh, K. Iyer, and N. Desai, "Feature Engineering Techniques for Climate Time-Series Forecasting," *Applied Artificial Intelligence*, 2024.

[10] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proceedings of the 22nd ACM SIGKDD Conference*, 2016.
[11] A. Sharma and V. Iyer, "Drought Risk Classification Using Rainfall Anomalies," *International Journal of Environmental Data Science*, 2024.
[12] F. Pedregosa et al., "Scikit-Learn: Machine Learning in Python," *Journal of Machine Learning Research*, 2011.
[13] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "Statistical and Machine Learning Forecasting Methods: A Comparative Study," *International Journal of Forecasting*, 2023.
[14] D. Alencar, M. Silva, and R. Santos, "Deploying Machine Learning Models Using FastAPI," *IEEE Software*, 2023.
[15] A. Ghosh and S. Dutta, "Analysis of Seasonal Rainfall Variability in India Using Long-Term IMD Data," *Climate Dynamics*, 2024.
[16] K. Rao and M. Kulkarni, "Feature Importance Analysis for Meteorological Prediction Systems," *International Journal of Data Science*, 2023.
[17] E. Breck et al., "The ML Test Score: A Rubric for Production Readiness," *Proceedings of the IEEE Big Data Conference*, 2017.
[18] P. Sculley et al., "Hidden Technical Debt in Machine Learning Systems," *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
[19] L. Biewald, "Experiment Tracking in Modern Machine Learning Systems," *Machine Learning Systems Workshop*, 2023.
[20] R. K. Patel and A. Malhotra, "Resilient MLOps Architectures for Climate Risk Assessment," *International Journal of AI and Climate Science*, 2025.

10.4 Further Reading

1. DVC – https://dvc.org
2. MLflow – https://mlflow.org
3. Prefect – https://www.prefect.io
4. GitHub Actions – https://docs.github.com/actions
5. Docker – https://docs.docker.com
6. Evidently AI – https://www.evidentlyai.com